Atelier — Création d'une Base de Données — Gestion de Bibliothèque

Par Robert DIASSÉ

Contexte

Une bibliothèque universitaire souhaite informatiser la gestion de ses ouvrages, de ses membres et de ses emprunts.

Elle a besoin de gérer :

- les **auteurs** des livres (nom, prénom, nationalité)
- les **catégories** de livres (roman, informatique, science...)
- les **livres** disponibles (titre, année de publication, auteur, catégorie)
- les **membres** inscrits (nom, prénom, email, date d'inscription)
- les **emprunts** réalisés (quel membre a emprunté quel livre, à quelle date)

Objectifs de l'atelier

1. Construire le **MCD** à partir du contexte métier
2. Dériver le **MLD** depuis le MCD
3. Créer la base de données **via le terminal MySQL**
4. Créer la même base de données **via phpMyAdmin**
5. Insérer des données et écrire des **requêtes SQL**

> ⚠ À chaque étape, une vérification est proposée avant de passer à la suivante.

Partie 1 — Modélisation : le MCD

Étape 1.1 — Identification des entités

À partir du contexte, on identifie les objets métier principaux. Chaque objet devient une **entité**.

Entité	Identifiant	Autres attributs
Auteurs	id_auteur	nom_auteur, prenom_auteur, nationalite

Entité	Identifiant	Autres attributs
Categories	id_categorie	nom_categorie
Livres	id_livre	titre, annee_publication
Membres	id_membre	nom_membre, prenom_membre, email, date_inscription
Emprunts	id_emprunt	date_emprunt, date_retour_prevue

> 💡 Un identifiant est toujours **unique**, **stable** et **minimal**.

Étape 1.2 — Identification des associations

On cherche les liens entre les entités en posant des questions métier :

- **Un auteur écrit plusieurs livres**, un livre est écrit par un seul auteur
  → Association **écrire** entre Auteurs (1,n) et Livres (1,1)
- **Un livre appartient à une seule catégorie**, une catégorie regroupe plusieurs livres
  → Association **appartenir** entre Categories (1,n) et Livres (1,1)
- **Un membre emprunte plusieurs livres**, un livre peut être emprunté par plusieurs membres (à des dates différentes)
  → Association **emprunter** entre Membres (0,n) et Livres (0,n)

> 💡 L'association **emprunter** est de type (n,n) : elle deviendra une table à part entière dans le MLD.

Étape 1.3 — Schéma MCD complet

```
┌─────────────────┐          écrire          ┌──────────────────────────┐
│   Auteurs       │   (1,n) ──────── (1,1)   │        Livres            │
├─────────────────┤                          ├──────────────────────────┤
│ id_auteur       │                          │ id_livre                 │
│ nom_auteur      │                          │ titre                    │
│ prenom_auteur   │                          │ annee_publication        │
│ nationalité     │                          └──────────────────────────┘
└─────────────────┘                                     │
                                                        │ (1,1)
                                             appartenir │
                                                        │ (1,n)
┌─────────────────┐       emprunter          ┌──────────────────────────┐
│   Membres       │   (0,n) ──────── (0,n)   │      Categories          │
├─────────────────┤                          ├──────────────────────────┤
│ id_membre       │                          │ id_categorie             │
│ nom_membre      │                          │ nom_categorie            │
│ prenom_membre   │                          └──────────────────────────┘
│ email           │
│ date_inscription│
└─────────────────┘
```

✅ Vérification MCD

Avant de passer au MLD, vérifiez les points suivants :

- ☐ Chaque entité possède un identifiant unique
- ☐ Les noms d'entités sont au **pluriel**
- ☐ Les noms d'associations sont des **verbes à l'infinitif**
- ☐ Les noms d'attributs sont au **singulier**
- ☐ Aucun attribut n'est calculable depuis d'autres attributs
- ☐ Aucune entité n'est dupliquée
- ☐ Les cardinalités sont cohérentes avec les règles métier

> Si tous les points sont cochés, vous pouvez passer au MLD.

Partie 2 — Du MCD au MLD

Rappel des règles de transformation

Cardinalité	Règle
(1,n) – (1,1)	La clé primaire du côté (1,n) **migre** vers l'entité du côté (1,1)
(n,n)	L'association devient une **table associative** avec les deux clés
(1,1) – (1,1)	Fusion des entités ou clé étrangère du côté optionnel

---

Étape 2.1 — Transformation des entités

Toute entité du MCD devient une **relation (table)** dans le MLD.
L'identifiant devient la **clé primaire** (noté `#` ou souligné).

```
Auteurs(#id_auteur, nom_auteur, prenom_auteur, nationalite)
Categories(#id_categorie, nom_categorie)
Livres(#id_livre, titre, annee_publication)
Membres(#id_membre, nom_membre, prenom_membre, email, date_inscription)
```

Étape 2.2 — Transformation des associations

Association écrire — (1,n) côté Auteurs, (1,1) côté Livres

La clé de Auteurs migre vers Livres :

```
Livres(#id_livre, titre, annee_publication, id_auteur, id_categorie)
```

Association appartenir — (1,n) côté Categories, (1,1) côté Livres

La clé de Categories migre aussi vers Livres :

```
Livres(#id_livre, titre, annee_publication, id_auteur, id_categorie)
```

Association emprunter — (0,n) – (0,n)

L'association devient une table associative :

```
Emprunts(#id_emprunt, id_livre, id_membre, date_emprunt, date_retour_prevue)
```

Étape 2.3 — Schéma relationnel (MLD final)

```
Auteurs(#id_auteur, nom_auteur, prenom_auteur, nationalite)

Categories(#id_categorie, nom_categorie)

Livres(#id_livre, titre, annee_publication, id_auteur, id_categorie)
    avec : id_auteur    → Auteurs.id_auteur
           id_categorie → Categories.id_categorie

Membres(#id_membre, nom_membre, prenom_membre, email, date_inscription)

Emprunts(#id_emprunt, id_livre, id_membre, date_emprunt, date_retour_prevue)
    avec : id_livre  → Livres.id_livre
           id_membre → Membres.id_membre
```

✅ Vérification MLD

Avant de créer la base, vérifiez :

- ☐ Chaque table possède une clé primaire
- ☐ Toutes les clés étrangères sont identifiées
- ☐ Les associations (n,n) ont bien été transformées en tables
- ☐ Aucune information du MCD n'a été perdue
- ☐ Le schéma est en **3FN** (pas de dépendance partielle ni transitive)

> Si tous les points sont cochés, vous pouvez créer la base de données.

Partie 3 — Création via le Terminal MySQL

Étape 3.1 — Configurer la variable d'environnement MySQL

Avant d'utiliser MySQL en ligne de commande, il faut que votre système sache où se trouve l'exécutable `mysql`.

Sur Windows

> 1. Faites un clic droit sur « **Ce PC** » → **Propriétés**
> 2. Cliquez sur **Paramètres système avancés**
> 3. Cliquez sur **Variables d'environnement**
> 4. Dans **Variables système**, sélectionnez `Path` puis cliquez sur **Modifier**
> 5. Ajoutez le chemin vers le dossier `bin` de MySQL, par exemple :
>
> ```
> C:\Program Files\MySQL\MySQL Server 8.0\bin
> ```
>
> 6. Cliquez sur **OK** pour valider

Sur Linux / macOS

Ouvrez un terminal et ajoutez la ligne suivante dans votre fichier de configuration shell :

```
# Pour Bash (Linux)
echo 'export PATH=$PATH:/usr/local/mysql/bin' >> ~/.bashrc
source ~/.bashrc

# Pour Zsh (macOS)
echo 'export PATH=$PATH:/usr/local/mysql/bin' >> ~/.zshrc
source ~/.zshrc
```

✅ Vérification de la variable d'environnement

Ouvrez un **nouveau terminal** (important : pour que les changements soient pris en compte) et tapez :

```
mysql --version
```

Vous devez voir quelque chose comme :

```
mysql  Ver 8.0.35 Distrib 8.0.35, for Win64 (x86_64)
```

> ❌ Si vous voyez `mysql n'est pas reconnu` (Windows) ou `command not found` (Linux/macOS), recommencez l'étape 3.1 et assurez-vous d'avoir ouvert un **nouveau** terminal.

Étape 3.2 — Se connecter à MySQL

```
mysql -u root -p
```

Saisissez votre mot de passe lorsqu'il est demandé.

Vous verrez l'invite MySQL :

```
mysql>
```

✅ Vérification de la connexion

Dans l'invite MySQL, tapez :

```
SHOW DATABASES;
```

Vous devez voir la liste des bases de données existantes, par exemple :

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
+--------------------+
```

> Si vous voyez cette liste, vous êtes bien connecté. Passez à l'étape suivante.

Étape 3.3 — Créer la base de données

```
CREATE DATABASE gestion_bibliotheque
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

> 💡 `utf8mb4` permet de stocker tous les caractères, y compris les accents et les emojis.

Puis sélectionnez la base :

```sql
USE gestion_bibliotheque;
```

✅ Vérification de la création de la base

```sql
SHOW DATABASES;
```

Vous devez voir `gestion_bibliotheque` dans la liste.

```sql
SELECT DATABASE();
```

Doit afficher :

```
+-----------------------+
| DATABASE()            |
+-----------------------+
| gestion_bibliotheque  |
+-----------------------+
```

Étape 3.4 — Créer les tables

> ⚠ **Important** : créez les tables dans l'ordre suivant pour respecter les clés étrangères.
> Les tables référencées doivent exister **avant** les tables qui les référencent.

Table Auteurs

```sql
CREATE TABLE Auteurs (
    id_auteur      INT          PRIMARY KEY AUTO_INCREMENT,
    nom_auteur     VARCHAR(50)  NOT NULL,
    prenom_auteur  VARCHAR(50)  NOT NULL,
    nationalite    VARCHAR(50)
);
```

Table Categories

```sql
CREATE TABLE Categories (
    id_categorie   INT          PRIMARY KEY AUTO_INCREMENT,
    nom_categorie  VARCHAR(50)  NOT NULL
);
```

Table Membres

```
CREATE TABLE Membres (
    id_membre        INT          PRIMARY KEY AUTO_INCREMENT,
    nom_membre       VARCHAR(50)  NOT NULL,
    prenom_membre    VARCHAR(50)  NOT NULL,
    email            VARCHAR(100),
    date_inscription DATE
);
```

Table Livres

```
CREATE TABLE Livres (
    id_livre           INT          PRIMARY KEY AUTO_INCREMENT,
    titre              VARCHAR(100) NOT NULL,
    annee_publication  INT,
    id_auteur          INT,
    id_categorie       INT,
    FOREIGN KEY (id_auteur)    REFERENCES Auteurs(id_auteur),
    FOREIGN KEY (id_categorie) REFERENCES Categories(id_categorie)
);
```

Table Emprunts

```
CREATE TABLE Emprunts (
    id_emprunt         INT  PRIMARY KEY AUTO_INCREMENT,
    id_livre           INT  NOT NULL,
    id_membre          INT  NOT NULL,
    date_emprunt       DATE NOT NULL,
    date_retour_prevue DATE,
    FOREIGN KEY (id_livre)  REFERENCES Livres(id_livre),
    FOREIGN KEY (id_membre) REFERENCES Membres(id_membre)
);
```

✅ Vérification des tables

Listez les tables créées :

```
SHOW TABLES;
```

Résultat attendu :

```
+-------------------------------+
| Tables_in_gestion_bibliotheque|
+-------------------------------+
| Auteurs                       |
| Categories                    |
| Emprunts                      |
| Livres                        |
| Membres                       |
+-------------------------------+
```

Vérifiez la structure d'une table :

```sql
DESCRIBE Livres;
```

```
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| id_livre         | int          | NO   | PRI | NULL    | auto_increment |
| titre            | varchar(100) | NO   |     | NULL    |                |
| annee_publication| int          | YES  |     | NULL    |                |
| id_auteur        | int          | YES  | MUL | NULL    |                |
| id_categorie     | int          | YES  | MUL | NULL    |                |
+------------------+--------------+------+-----+---------+----------------+
```

> `PRI` = clé primaire, `MUL` = clé étrangère. Si vous voyez cela, vos tables sont correctement créées.

Vérifiez aussi les contraintes de clés étrangères :

```sql
SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'gestion_bibliotheque'
  AND REFERENCED_TABLE_NAME IS NOT NULL;
```

Partie 4 — Création via phpMyAdmin

> Cette partie vous montre comment réaliser les mêmes opérations via l'interface graphique de phpMyAdmin. C'est le chemin le plus visuel pour débutants.

Étape 4.1 — Accéder à phpMyAdmin

1. Démarrez votre serveur local (**XAMPP**, **WAMP**, **MAMP** ou **Laragon** selon votre installation)
2. Ouvrez votre navigateur et allez à : `http://localhost/phpmyadmin`
3. Connectez-vous avec vos identifiants (souvent `root` sans mot de passe par défaut)

Étape 4.2 — Créer la base de données

> 1. Dans la colonne de gauche, cliquez sur **« Nouvelle base de données »**
> 2. Dans le champ **Nom de la base de données**, tapez : `gestion_bibliotheque`
> 3. Dans le menu déroulant **Interclassement**, choisissez : `utf8mb4_unicode_ci`
> 4. Cliquez sur le bouton **Créer**

✅ Vérification (phpMyAdmin)

Dans la colonne de gauche, vous devez voir `gestion_bibliotheque` apparaître dans la liste des bases de données.

Cliquez dessus pour la sélectionner. La zone centrale affiche :

```
Aucune table trouvée dans la base de données.
```

C'est normal : la base est vide pour l'instant.

Étape 4.3 — Créer les tables via l'onglet SQL

phpMyAdmin propose deux méthodes pour créer des tables :

- **Via l'interface graphique** (formulaire avec cases à remplir)
- **Via l'onglet SQL** (on écrit le SQL directement — recommandé car plus rapide)

> Nous utilisons l'onglet **SQL**, qui vous habitue à écrire vous-même le SQL.

Procédure

> 1. Sélectionnez la base `gestion_bibliotheque` dans la colonne gauche
> 2. Cliquez sur l'onglet **SQL** en haut de la page
> 3. Copiez-collez le bloc SQL suivant dans la zone de texte
> 4. Cliquez sur **Exécuter**

```sql
CREATE TABLE Auteurs (
    id_auteur      INT           PRIMARY KEY AUTO_INCREMENT,
    nom_auteur     VARCHAR(50)   NOT NULL,
    prenom_auteur  VARCHAR(50)   NOT NULL,
    nationalite    VARCHAR(50)
);
```

```sql
CREATE TABLE Categories (
    id_categorie  INT         PRIMARY KEY AUTO_INCREMENT,
    nom_categorie VARCHAR(50) NOT NULL
);

CREATE TABLE Membres (
    id_membre        INT          PRIMARY KEY AUTO_INCREMENT,
    nom_membre       VARCHAR(50)  NOT NULL,
    prenom_membre    VARCHAR(50)  NOT NULL,
    email            VARCHAR(100),
    date_inscription DATE
);

CREATE TABLE Livres (
    id_livre           INT          PRIMARY KEY AUTO_INCREMENT,
    titre              VARCHAR(100) NOT NULL,
    annee_publication  INT,
    id_auteur          INT,
    id_categorie       INT,
    FOREIGN KEY (id_auteur)    REFERENCES Auteurs(id_auteur),
    FOREIGN KEY (id_categorie) REFERENCES Categories(id_categorie)
);

CREATE TABLE Emprunts (
    id_emprunt         INT  PRIMARY KEY AUTO_INCREMENT,
    id_livre           INT  NOT NULL,
    id_membre          INT  NOT NULL,
    date_emprunt       DATE NOT NULL,
    date_retour_prevue DATE,
    FOREIGN KEY (id_livre)  REFERENCES Livres(id_livre),
    FOREIGN KEY (id_membre) REFERENCES Membres(id_membre)
);
```

✅ Vérification (phpMyAdmin)

Après exécution, phpMyAdmin affiche un message vert :

```
Requête SQL exécutée avec succès.
```

Dans la colonne de gauche, dépliez `gestion_bibliotheque`. Vous devez voir :

- Auteurs
- Categories
- Emprunts
- Livres
- Membres

Cliquez sur **Livres** puis sur l'onglet **Structure** pour visualiser les colonnes et les clés.

> L'icône 🔑 indique la clé primaire. Les colonnes avec une flèche indiquent les clés étrangères.

Partie 5 — Insertion de données

> Cette partie est identique que vous utilisiez le terminal ou phpMyAdmin (onglet SQL).

Étape 5.1 — Insérer les auteurs

```sql
INSERT INTO Auteurs (nom_auteur, prenom_auteur, nationalite) VALUES
('Diallo',  'Mamadou',  'Sénégalaise'),
('Kane',    'Cheikh Hamidou', 'Sénégalaise'),
('Bâ',      'Mariama',  'Sénégalaise'),
('Orwell',  'George',   'Britannique'),
('Asimov',  'Isaac',    'Américaine');
```

Étape 5.2 — Insérer les catégories

```sql
INSERT INTO Categories (nom_categorie) VALUES
('Roman'),
('Informatique'),
('Science-Fiction'),
('Philosophie'),
('Histoire');
```

Étape 5.3 — Insérer les livres

```sql
INSERT INTO Livres (titre, annee_publication, id_auteur, id_categorie) VALUES
('L\'Aventure ambiguë',     1961, 2, 1),
('Une si longue lettre',    1979, 3, 1),
('1984',                    1949, 4, 1),
('Fondation',               1951, 5, 3),
('Le Robot',                1950, 5, 3),
('Introduction aux SGBD',   2020, 1, 2),
('Les Bouts de bois de Dieu',1960,2, 1),
('La Ferme des animaux',    1945, 4, 1);
```

Étape 5.4 — Insérer les membres

```sql
INSERT INTO Membres (nom_membre, prenom_membre, email, date_inscription) VALUES
('Sow',    'Fatou',     'fatou.sow@isi.sn',    '2024-09-01'),
('Diop',   'Ousmane',   'ousmane.diop@isi.sn', '2024-09-01'),
```

```
('Ndiaye', 'Aissatou',  'aissatou.n@isi.sn',   '2024-09-15'),
('Fall',   'Ibrahima',  'ibrahima.fall@isi.sn', '2024-10-01'),
('Cissé',  'Marième',   'marieme.c@isi.sn',    '2024-10-01');
```

Étape 5.5 — Insérer les emprunts

```sql
INSERT INTO Emprunts (id_livre, id_membre, date_emprunt, date_retour_prevue)
VALUES
(1, 1, '2025-01-10', '2025-01-24'),
(3, 1, '2025-01-15', '2025-01-29'),
(4, 2, '2025-01-12', '2025-01-26'),
(2, 3, '2025-01-20', '2025-02-03'),
(5, 3, '2025-01-20', '2025-02-03'),
(6, 4, '2025-02-01', '2025-02-15'),
(1, 5, '2025-02-05', '2025-02-19'),
(7, 2, '2025-02-10', '2025-02-24');
```

✅ Vérification des insertions

Vérifiez le contenu de chaque table :

```sql
SELECT * FROM Auteurs;
SELECT * FROM Categories;
SELECT * FROM Livres;
SELECT * FROM Membres;
SELECT * FROM Emprunts;
```

Vérifiez le nombre de lignes dans chaque table :

```sql
SELECT 'Auteurs'    AS table_name, COUNT(*) AS nb_lignes FROM Auteurs
UNION
SELECT 'Categories',               COUNT(*)               FROM Categories
UNION
SELECT 'Livres',                   COUNT(*)               FROM Livres
UNION
SELECT 'Membres',                  COUNT(*)               FROM Membres
UNION
SELECT 'Emprunts',                 COUNT(*)               FROM Emprunts;
```

Résultat attendu :

```
+------------+-----------+
| table_name | nb_lignes |
```

```
+------------+-----------+
| Auteurs    |         5 |
| Categories |         5 |
| Livres     |         8 |
| Membres    |         5 |
| Emprunts   |         8 |
+------------+-----------+
```

Partie 6 — Requêtes SQL

> Pour chaque requête, lisez d'abord la question, essayez d'écrire la requête seul(e), puis comparez avec la solution proposée.

6.1 — Requêtes simples (sur une seule table)

Question 1 : Afficher tous les livres publiés après 1960

```sql
SELECT titre, annee_publication
FROM Livres
WHERE annee_publication > 1960;
```

Question 2 : Afficher les membres inscrits en 2024, triés par nom

```sql
SELECT nom_membre, prenom_membre, date_inscription
FROM Membres
WHERE YEAR(date_inscription) = 2024
ORDER BY nom_membre ASC;
```

Question 3 : Combien de livres y a-t-il dans la base ?

```sql
SELECT COUNT(*) AS nombre_de_livres
FROM Livres;
```

6.2 — Requêtes avec jointure (plusieurs tables)

Question 4 : Afficher le titre des livres avec le nom de leur auteur

```sql
SELECT L.titre, A.nom_auteur, A.prenom_auteur
FROM Livres L
JOIN Auteurs A ON L.id_auteur = A.id_auteur;
```

Question 5 : Afficher les livres avec leur catégorie et leur auteur

```sql
SELECT L.titre, A.nom_auteur, C.nom_categorie
FROM Livres L
JOIN Auteurs    A ON L.id_auteur    = A.id_auteur
JOIN Categories C ON L.id_categorie = C.id_categorie
ORDER BY C.nom_categorie, L.titre;
```

Question 6 : Quels membres ont emprunté quel livre, et à quelle date ?

```sql
SELECT M.nom_membre, M.prenom_membre, L.titre, E.date_emprunt,
E.date_retour_prevue
FROM Emprunts E
JOIN Membres M ON E.id_membre = M.id_membre
JOIN Livres  L ON E.id_livre  = L.id_livre
ORDER BY E.date_emprunt;
```

Question 7 : Quels livres ont été empruntés par Fatou Sow ?

```sql
SELECT L.titre, E.date_emprunt
FROM Emprunts E
JOIN Livres  L ON E.id_livre  = L.id_livre
JOIN Membres M ON E.id_membre = M.id_membre
WHERE M.nom_membre = 'Sow' AND M.prenom_membre = 'Fatou';
```

6.3 — Requêtes avec agrégation

Question 8 : Combien d'emprunts chaque membre a-t-il effectué ?

```sql
SELECT M.nom_membre, M.prenom_membre, COUNT(E.id_emprunt) AS nb_emprunts
FROM Membres M
LEFT JOIN Emprunts E ON M.id_membre = E.id_membre
GROUP BY M.id_membre, M.nom_membre, M.prenom_membre
ORDER BY nb_emprunts DESC;
```

> 💡 `LEFT JOIN` permet d'afficher aussi les membres qui n'ont aucun emprunt (résultat = 0).

Question 9 : Quel est le livre le plus emprunté ?

```sql
SELECT L.titre, COUNT(E.id_emprunt) AS nb_emprunts
FROM Livres L
LEFT JOIN Emprunts E ON L.id_livre = E.id_livre
GROUP BY L.id_livre, L.titre
ORDER BY nb_emprunts DESC
LIMIT 1;
```

Question 10 : Combien de livres chaque catégorie contient-elle ?

```sql
SELECT C.nom_categorie, COUNT(L.id_livre) AS nb_livres
FROM Categories C
LEFT JOIN Livres L ON C.id_categorie = L.id_categorie
GROUP BY C.id_categorie, C.nom_categorie
ORDER BY nb_livres DESC;
```

✅ Vérification finale

Exécutez la requête suivante pour vous assurer que toutes les jointures fonctionnent correctement :

```sql
SELECT
    M.nom_membre,
    M.prenom_membre,
    L.titre,
    A.nom_auteur,
    C.nom_categorie,
    E.date_emprunt
FROM Emprunts E
JOIN Membres    M ON E.id_membre    = M.id_membre
JOIN Livres     L ON E.id_livre     = L.id_livre
JOIN Auteurs    A ON L.id_auteur    = A.id_auteur
JOIN Categories C ON L.id_categorie = C.id_categorie
ORDER BY E.date_emprunt;
```

Si vous obtenez 8 lignes avec toutes les colonnes remplies, votre base est opérationnelle. 🎉

Recommandations — Pour aller plus loin

🔒 Sécurité

1. Ne jamais utiliser l'utilisateur `root` en production

Le compte `root` a tous les droits. En production, créez un utilisateur dédié avec des droits limités :

```
-- Créer un utilisateur
CREATE USER 'biblio_user'@'localhost' IDENTIFIED BY 'MotDePasseFort!2025';

-- Lui donner uniquement les droits nécessaires sur la base
GRANT SELECT, INSERT, UPDATE ON gestion_bibliotheque.* TO
'biblio_user'@'localhost';

-- Appliquer les changements
FLUSH PRIVILEGES;
```

---

2. Protéger les mots de passe

Ne stockez **jamais** un mot de passe en clair dans la base de données. Utilisez toujours une fonction de hachage :

```
-- Exemple avec SHA2 (256 bits) — à faire côté application en pratique
INSERT INTO Membres (nom_membre, prenom_membre, email, mot_de_passe)
VALUES ('Diop', 'Omar', 'omar@isi.sn', SHA2('monmotdepasse', 256));
```

> 💡 En pratique, le hachage se fait côté code applicatif (PHP, Python…), pas en SQL.

---

3. Sauvegarder régulièrement la base

```
# Sauvegarder la base depuis le terminal
mysqldump -u root -p gestion_bibliotheque > sauvegarde_$(date +%Y%m%d).sql

# Restaurer une sauvegarde
mysql -u root -p gestion_bibliotheque < sauvegarde_20250328.sql
```

🚀 Améliorations fonctionnelles

4. Ajouter des index pour accélérer les requêtes

Les clés étrangères créent automatiquement des index, mais vous pouvez en ajouter sur les colonnes souvent recherchées :

```sql
-- Accélérer les recherches par titre
CREATE INDEX idx_titre ON Livres(titre);

-- Accélérer les recherches par date d'emprunt
CREATE INDEX idx_date_emprunt ON Emprunts(date_emprunt);
```

---

5. Ajouter des contraintes métier

```sql
-- S'assurer que l'année de publication est réaliste
ALTER TABLE Livres
ADD CONSTRAINT chk_annee
CHECK (annee_publication BETWEEN 1000 AND YEAR(CURDATE()));

-- S'assurer que la date de retour est après la date d'emprunt
ALTER TABLE Emprunts
ADD CONSTRAINT chk_dates
CHECK (date_retour_prevue > date_emprunt);
```

---

6. Créer des vues pour simplifier les requêtes répétitives

Une vue est une requête sauvegardée que vous pouvez utiliser comme une table :

```sql
CREATE VIEW vue_emprunts_detail AS
SELECT
    E.id_emprunt,
    CONCAT(M.prenom_membre, ' ', M.nom_membre) AS membre,
    L.titre,
    CONCAT(A.prenom_auteur, ' ', A.nom_auteur) AS auteur,
    E.date_emprunt,
    E.date_retour_prevue
FROM Emprunts E
JOIN Membres    M ON E.id_membre    = M.id_membre
JOIN Livres     L ON E.id_livre     = L.id_livre
JOIN Auteurs    A ON L.id_auteur    = A.id_auteur;

-- Utilisation aussi simple qu'une table
SELECT * FROM vue_emprunts_detail WHERE membre LIKE '%Sow%';
```

🔄 Changer de SGBD

Le SQL est standardisé, mais chaque SGBD a ses particularités. Voici comment migrer :

Ce que vous avez fait	MySQL	PostgreSQL	SQLite
Créer une base	`CREATE DATABASE`	`CREATE DATABASE`	Créer un fichier `.db`
Auto-incrément	`AUTO_INCREMENT`	`SERIAL` ou `GENERATED ALWAYS AS IDENTITY`	`AUTOINCREMENT`
Type texte	`VARCHAR(n)`	`VARCHAR(n)` ou `TEXT`	`TEXT`
Afficher tables	`SHOW TABLES`	`\dt`	`.tables`
Sauvegarder	`mysqldump`	`pg_dump`	Copier le fichier `.db`

> 💡 **PostgreSQL** est souvent préféré en production pour sa robustesse et sa conformité au standard SQL. L'essentiel du SQL que vous avez appris ici fonctionne sans modification.

📚 Pour continuer à apprendre

Sujet	Ce que c'est	Pourquoi l'apprendre
Transactions	`BEGIN`, `COMMIT`, `ROLLBACK`	Garantir la cohérence lors d'opérations multiples
Procédures stockées	Code SQL stocké dans la base	Automatiser des traitements répétitifs
Triggers	Déclencheurs automatiques	Agir automatiquement lors d'INSERT/UPDATE/DELETE
ORM	Couche d'abstraction (ex. SQLAlchemy)	Manipuler la base depuis Python sans écrire du SQL
NoSQL	MongoDB, Redis...	Gérer des données non structurées ou très volumineuses
Réplication	Synchronisation entre serveurs	Haute disponibilité et tolérance aux pannes

---

✨ Récapitulatif de l'atelier

```
Contexte métier
      │
      ▼
   MCD (entités, associations, cardinalités)
      │
      ▼
   MLD (tables, clés primaires, clés étrangères)
      │
      ▼
   SQL — CREATE DATABASE / CREATE TABLE
```

```
        │
     ┌──┴──┐
  Terminal  phpMyAdmin
        │
        ▼
     INSERT INTO (données)
        │
        ▼
     SELECT / JOIN / GROUP BY (requêtes)
        │
        ▼
     Sécurité, index, vues, sauvegarde...
```

> Vous savez maintenant concevoir une base de données de A à Z. La prochaine étape : connecter cette base à une application web (PHP, Python, Node.js...).